Homework 3: Learning on Sequence Data
CPE 487/587

Instructor: Rahul Bhadani

Due: Mar 05, 2026, 11:59 PM
147 Points

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

Submission instruction:

Upload a .pdf on Canvas with the format `CPE487587-LastFirst-HW-XX`. For example, if your name is Sam Wells, your file name should be `CPE487587-LastFirst-HW-XX.pdf`. If there is a programming assignment, then you should include your source code along with your PDF files in a zip file `CPE487587-LastFirst-HW-XX.zip`. Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

All CPE 587 students are required to submit their response in a Latex-generated PDF.

Provide a PDF with a solution to your theory portion and github commit hash and GitHub repo URL for your practice portion. In addition, include any plots in the PDF for practice portion if asked.

Theory

1 Feature Reduction in CNN (10 Points)

1. Consider a 6×6 input matrix. If we apply a kernel of size 3×3 with a stride of 1 and padding of 0, what's the dimension of the output? **(3 Points)**

2. Consider an input matrix

$$I = \begin{bmatrix} 7 & 2 & 1 & 11 & 5 & 4 \\ 8 & 2 & 3 & 4 & 3 & 4 \\ 5 & 5 & 2 & 8 & 10 & 0 \\ 8 & 8 & 1 & 7 & 3 & 5 \\ 8 & 5 & 5 & 8 & 11 & 7 \\ 5 & 0 & 5 & 7 & 4 & 9 \end{bmatrix}$$

with a kernel as

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

With a stride of 1 and padding of zero, compute the output. **(7 Points)**

1.1 Solution

1. 4×4.

2.

$$O = \begin{bmatrix} 2-3 & 3-4 & 4-3 & 3-4 \\ 5-2 & 2-8 & 8-10 & 10-0 \\ 8-1 & 1-7 & 7-3 & 3-5 \\ 5-5 & 5-8 & 8-11 & 11-7 \end{bmatrix} = \begin{bmatrix} -1 & -1 & 1 & -1 \\ 3 & -6 & -2 & 10 \\ 7 & -6 & 4 & -2 \\ 0 & -3 & -3 & 4 \end{bmatrix}$$

2 Downsampling (2 Pooints)

Which operation in the CNN causes downsampling of the input?

2.1 Solution

Pooling.

3 Understanding CNN layers (10 Points)

Consider a convolutional layer:

```
conv = nn.Conv2d(in_channels=4,
                 out_channels=6,
                 kernel_size=3,
                 stride=1,
                 padding=0)
```

1. Provide an example Tensor dimension that may be suitable to feed into the above convolution layer. **(2 Points)**
2. Does padding parameter affect input Tensor dimension of the convolution layer? **(2 Points)**
3. If we consider bias, how many trainable parameter the above Conv2d layer has? **(3 Points)**
4. If we have input RGB image of size 20×20 and batch size of 5 , what would be the output dimension of Conv2d defined above? **(3 Points)**

3.1 Solution

1. Input should be

   batchsize $\times$ channels $\times$ height $\times$ width.

   One such input can be $2 \times 4 \times 64 \times 128$.

2. No. It only affects the output dimension.

3. Number of parameters $P = (C_{in} \times K_h \times K_w + 1)C_{out}$.

   which is

   $P = (4 \times 3 \times 3 + 1) \times 6 = 37 \times 6 = 222$.

4. For the given input dimension d,

   output dimension

$$e = \left\lfloor \frac{d + 2p - k}{s} + 1 \right\rfloor$$

where p is padding, k is kernel size, and s is stride.

Hence

$$e = \left\lfloor \frac{20 - 3}{1} + 1 \right\rfloor = 18$$

Hence output dimension of the image is 18×18 and the output dimension of the tensor is $5 \times 6 \times 18 \times 18$.

4 n-Gram Language Model (10 Points)

Given a corpus: **"I like apple. I like banana. I like apple and banana."** Assume all punctuation marks are removed, and the corpus is split into words as: [I, like, apple, I, like, banana, I, like, apple, and, banana].

1. Calculate the unigram probability $P(like)$. **(2 Points)**
2. Using the same corpus, calculate the bigram probability $P(like|I)$? **(4 Points)**
3. Using the same corpus, calculate $P(banana|apple)$. **(4 Points)**

4.1 Solution

1. $P(like) = \frac{3}{11}$
2. $P(like|I) = \frac{3}{3} = 1.$
3. $P(banana|apple) = 0$ as no occurence of banana is there after apple in the given corpus.

5 RNN Cell (15 Points)

Consider a single recurrent neural network (RNN) cell. The cell has an input x_t at time step t, a hidden state h_{t-1} from the previous time step, and computes the new hidden state h_t and output y_t. The cell uses the following formulas to update its state and compute the output:

$$h_t = \sigma(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

$$y_t = W_y \cdot h_t + b_y$$

where W_h and W_x are weight matrices for the hidden state and input, respectively, b_h is the bias for the hidden state, W_y is the weight matrix for the output, b_y is the bias for the output. h_t formula contains recurrent connection as we saw in the class. σ is the sigmoid activation function.

Given:

1. $$W_h = \begin{bmatrix} 0.5 & -0.6 \\ 0.8 & -0.1 \end{bmatrix} \tag{1}$$
2. $$W_x = \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix} \tag{2}$$
3. $$b_h = \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} \tag{3}$$
4. $$W_y = \begin{bmatrix} 0.3 & -0.7 \end{bmatrix} \tag{4}$$
5. $$b_y = [0.1] \tag{5}$$
6. $$h_{t-1} = \begin{bmatrix} 0.7 \\ -0.8 \end{bmatrix} \tag{6}$$
7. $$x_t = [0.5] \tag{7}$$

Calculate h_t and y_t for time step t.

5.1 Solution

$$\begin{aligned} W_h \cdot h_{t-1} + W_x \cdot x_t + b_h &= \begin{bmatrix} 0.5 & -0.6 \\ 0.8 & -0.1 \end{bmatrix} \times \begin{bmatrix} 0.7 \\ -0.8 \end{bmatrix} + \begin{bmatrix} -0.2 \\ 0.4 \end{bmatrix} \times [0.5] + \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} + \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} \\ &= \\ &= \begin{bmatrix} 0.83 \\ 0.64 \end{bmatrix} + \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} + \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} \\ &= \begin{bmatrix} 0.83 \\ 0.64 \end{bmatrix} + \begin{bmatrix} -0.20 \\ 0.40 \end{bmatrix} = \begin{bmatrix} 0.63 \\ 1.04 \end{bmatrix} \end{aligned} \tag{8}$$

$$h_t = \sigma(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h) = \begin{bmatrix} \sigma(0.63) \\ \sigma(1.04) \end{bmatrix} = \begin{bmatrix} \frac{1}{1+\exp(-0.63)} \\ \frac{1}{1+\exp(-1.04)} \end{bmatrix} = \begin{bmatrix} 0.652 \\ 0.739 \end{bmatrix} \tag{9}$$

$$\begin{aligned} y_t = W_y \cdot h_t + b_y &= \begin{bmatrix} 0.3 & -0.7 \end{bmatrix} \times \begin{bmatrix} 0.652 \\ 0.739 \end{bmatrix} + [0.1] \\ &= 0.1956 - 0.5173 + 0.1 = -0.2217 \end{aligned} \tag{10}$$

Practice

You must update README with an instruction on how to make inferences using your trained .onnx file with unseen dataset for both models developed as a part of this assignment.

6 Convolutional Neural Network for Imagenet (50 Points)

Imagenet dataset is available on HuggingFace https://huggingface.co/datasets/ILSVRC/imagenet-1k that I have downloaded in Lovelace machine in the location `/data/CPE_487-587/imagenet-1k`.

To read the ImageNet-1k dataset from cached Arrow files, we can use the Hugging Face `datasets` library with the `load_dataset()` function.

We need to specify the `cache_dir` parameter to point to the parent directory containing the `"ILSVRC___imagenet-1k"` folder (e.g., `cache_dir="/data/CPE_487-587/imagenet-1k"`).

This allows you to access the pre-downloaded dataset without re-downloading, where each split (train, validation, test) can be accessed as `dataset['train']`, `dataset['validation']`, or `dataset['test']`.

Each example contains an `'image'` field (a PIL Image object) and a `'label'` field (an integer from 0–999 representing one of 1000 classes). The labels are based on WordNet synsets, meaning each label string contains multiple comma-separated synonyms for the same concept (e.g., `"plane, carpenter's plane, woodworking plane"`), which includes common names, alternative names, and sometimes scientific names.

For practical use, you can extract the primary class name by taking the first word before the comma using `class_names[label_id].split(',')[0]`, where `class_names` is obtained from `train_dataset.features['label'].names`.

In your script, you can read the already downloaded data as follows:

```
# Load dataset
dataset = load_dataset(
    "ILSVRC/imagenet-1k",
    cache_dir="/data/CPE_487-587/imagenet-1k"
)

train_dataset = dataset['train']
```

```
val_dataset = dataset['validation']
num_classes = len(train_dataset.features['label'].names)
print(f"Number of classes: {num_classes}")
```

To select a subset of training and validation sample, you may use `select` function

```
train_size = int(len(dataset['train']) * 0.10) % 10 percent selection
val_size = int(len(dataset['validation']) * 0.05) % 5 percent selection

train_dataset = dataset['train'].select(range(train_size))
val_dataset = dataset['validation'].select(range(val_size))
```

It is useful to select small portion during the training stage.

Following code can be used for displaying a sample image:

```
first_example = train_dataset[0]
image = first_example['image']
label_id = first_example['label']

full_label = class_names[label_id]
primary_name = full_label.split(',')[0].strip()

plt.figure(figsize=(8, 8))
plt.imshow(image)
plt.title(f"ID {label_id}: {primary_name}\n({full_label})", fontsize=10)
plt.axis('off')
plt.show()
```

At the same time, we also need to transform images for variability and apply transforms to training and validation dataset. Validation transform is slightly different to avoid overfitting.

```
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

# Apply transforms
def preprocess_train(examples):
    images = [train_transform(img.convert('RGB')) for img in examples['image']]
    labels = examples['label']

    return {
        'pixel_values': images,
        'labels': labels
    }


def preprocess_val(examples):
```

```
    images = [val_transform(img.convert('RGB')) for img in examples['image']]
    labels = examples['label']

    return {
        'pixel_values': images,
        'labels': labels
    }

train_dataset = train_dataset.with_transform(preprocess_train)
val_dataset = val_dataset.with_transform(preprocess_val)

def collate_fn(batch):
    # Extract pixel_values and labels from each item
    pixel_values = torch.stack([item['pixel_values'] for item in batch])
    labels = torch.tensor([item['labels'] for item in batch])

    return {
        'pixel_values': pixel_values,
        'labels': labels
    }

# Create DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,
    pin_memory=True,  # Important for faster GPU transfer
    collate_fn=collate_fn
)

val_loader = DataLoader(
    val_dataset,
    batch_size=128,
    shuffle=False,
    pin_memory=True,
)
```

Also, Lovelace machine has a total of eight GPUs, in order to make out most out of it, you should choose GPU that is the least utilized at the moment. You can use the following code to decide which GPU device to use:

```
import subprocess
import torch

def get_best_gpu(strategy="utilization"):
    """
    Select best GPU by 'utilization' or 'memory'.
    """
    if strategy == "memory":
        # Use PyTorch directly for free memory
        free_mem = []
        for i in range(torch.cuda.device_count()):
            props = torch.cuda.mem_get_info(i)  # (free, total)
            free_mem.append(props[0])
        return free_mem.index(max(free_mem))

    elif strategy == "utilization":
        result = subprocess.run(
            ["nvidia-smi", "--query-gpu=utilization.gpu", "--format=csv,noheader,nounits"],
            capture_output=True, text=True
        )
        utilizations = [int(x.strip()) for x in result.stdout.strip().split("\n")]
        return utilizations.index(min(utilizations))
```

```
23
24  # Pick strategy: "utilization" or "memory"
25  device_id = get_best_gpu(strategy="utilization")
26  device = torch.device(f"cuda:{device_id}")
27  print(f"Selected GPU: {device_id}")
```

6.1 Tasks Description and Deliverables

Your job is create Convolutional Neural Network model to create an image classifier based on the Imagenet dataset.

You are required to implement the following CNN model given in Figure 1.

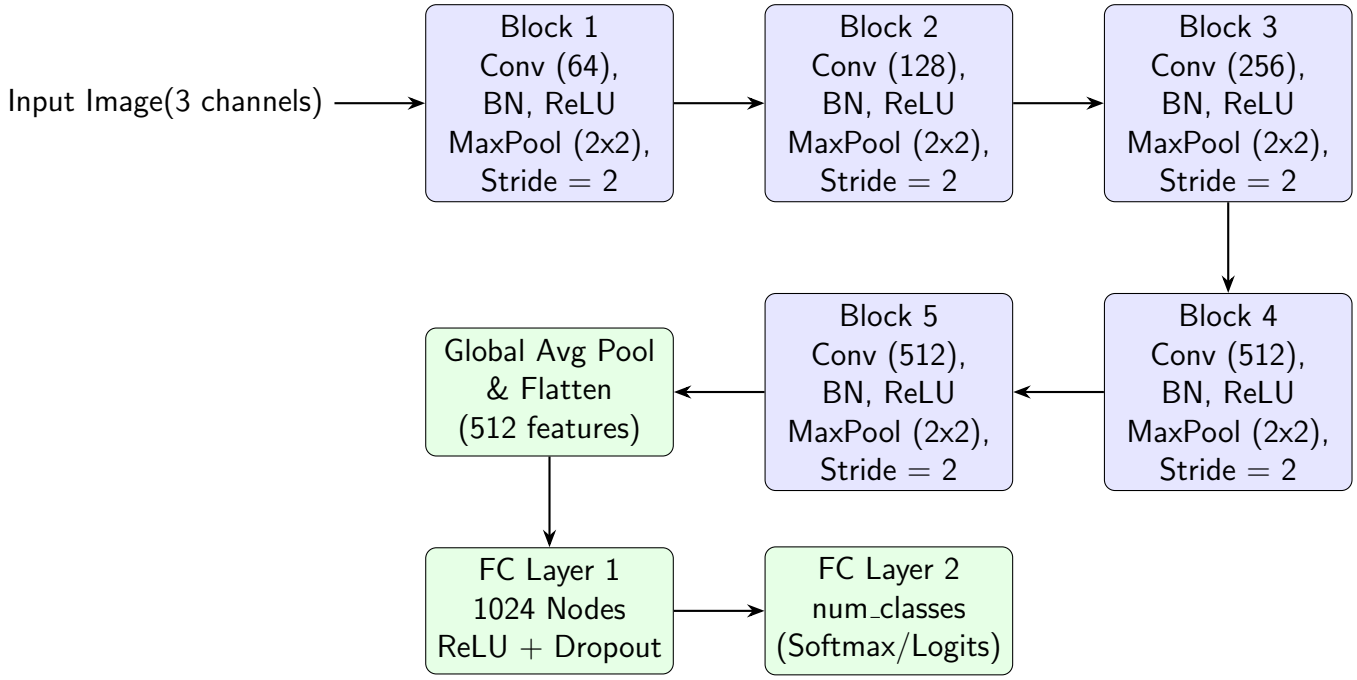

Figure 1: CNN Architecture, BN means Batch Normalization, FC means fully connected layer. Global average pooling can be implemented using `nn.AdaptiveAvgPool2d`. Conv layers have stride of 1 with padding of 1, and maxpool layer has stride of 2.

6.2 Deliverables (42 Points)

1. In `deepl` subpackage, you already have a module multiclass.py after HW02. There you should define a custom composite layer called `ConvLayer` corresponding to Block defined in Figure 1. The layer should be able to take number of input channels and output channels as input argument so that same class can be used to instantiate Block 1 to 5 shown in Figure 1.

2. Use `ConvLayer` to create `ImageNetCNN` class that appropriate put together various layers to construct CNN architecture shown in Fiigure 1.

3. You should implement `CNNTrainer` class similar to how you implemented in Homework 02. However your training function should be able to use training and validation DataLoader as specified above. Your training function should print epoch number for every tenth batch every epoch, loss and training accuracy, and validation accuracy.

   You can exercise some freedom in how you want to implement `CNNTrainer` but overall structure will be very similar to what you did in Homework 02. You should be able to save trained model as ONNX as well. Do not forget to modify your `__init__.py`

4. In your scripts folder, create a file called `imagenet_impl.py` that will be importing your classes defined and perform the following task

   (a) Read Imagenet data

   (b) Perform image transforms and create training and validation dataloader

   (c) Save an example training and validation image in the script folder. Instantiate your `ImageNetCNN` and `CNNTrainer` classes.

   (d) You should use cross entropy loss, SGD optimizer with learning rate of 0.01, momentum of 0.9, and weight decay of 1e-4. Use step learning rate scheduler with step size of 30 and gamma value of 0.1.

   (e) You should be able to write your script in such a way that it should take command line argument such as number of epochs to train, ratio of training dataset and validation dataset to use for training.

   (f) At the end of the training, a plot should be saved that should display loss and accuracy as a function of epochs. Commit the plot to the GitHub and provide the file in PDF submission.

5. Write a helping bash script `imagenet_impl.sh` to execute the training in the background. You don't need to train your image classifier multiple times, but command should have values of number of epochs, and ratio of training dataset and validation dataset to use for training. Supply number of epochs as 10000 at least.

6. Write a Python script called `imagenet_inference.py` that loads the trained model, take a new image as an argument and predict what class this image belongs to. Commit the example image, and trained model .onnx to your GitHub.

6.3 Questions (2 Each)

1. What is the original number of training samples in the dataset?

2. What is the original number of validation samples in the dataset?

3. How many number of classes are present in the dataset?

4. What is the total number of trainable parameters in your CNN model?

6.4 Solution

1. 1281167
2. 50000
3. 1000
4. 5464040

7 Cruise Control State Classifier (50 Points)

Your goal is create a suprvised classifier to predict the adaptive cruise control status of a car based on the speed data. Valid ACC status are described as:

Value	State Name	Description
0	`off`	System Inactive. Main switch is off or ignition cycled.
2	`disabled`	Standby. Main switch ON, but no speed set (cancelled).
5	`faulted`	System Error. Sensor blocked or hardware failure.
6	`enabled`	Active cruising (maintaining speed or following).
10	`hold_waiting_user_cmd`	Long Standstill (> 3s). Holds brakes; requires driver input to move.
11	`hold`	Short Standstill (< 3s). Holds brakes; resumes automatically.

Here is the machine learning problem: based on the speed data, determine if ACC is enabled or not. (6 or otherwise).

For that you will create a class called `ACCNet` in a module `acc_classifier.py` in `deepl` subpackage.

To refer to data for this problem, please to the folder `/data/CPE_487-587/ACCDataset`. It has multiple .csv files.

You will only need to use those csv files that ends with `decoded_wheel_speed_fl.csv` along with labels in `acc_status.cv`. You will need to binarize the labels as 0 for not ACC and 1 for ACC (which is originally 6 in the above table).

Each of wheel speed fl(fl = front left) csv file names start with a timestamp representing different driving experiment.

In addition, `decoded_wheel_speed_fl.csv` contains Time column which is in seconds and Message column that has units in km/h that would you need to convert to m/s. Similarly,

`acc_status.cv` has Time column with values in seconds and Message column that has ACC status. You will need to use any other columns in these two files.

Here is another problem: sampling rate of speed and ACC status are not the same. Hence number of rows in both datasets are not the same. Clearly in that case number of feature samples and number of labels are not the same. In such a case, you can use **Zero-Order Hold technique** to upsample. See https://en.wikipedia.org/wiki/Zero-order_hold for more details.

1. Create a PyTorch Dataset class and PyTorch Dataloader for batch training that takes wheel speed files as input and create multiple feature columns based on historical speed values.

   For example, originally, relevant column is Message for `decoded_wheel_speed_fl.csv`. Call it v_t Your PyTorch Dataset class will take all necessary .csv files as inputs and create a dataset with columns v_t, v_{t-1}, upto v_{t-k}. Choose the value of $k = 10$.

   Your neural network will essentialy be an abstraction of the following model:

   $$\hat{y}_t = f(\{v_{t-i}\}_{i=0}^{k}; \mathbf{W})$$

   where $\mathbf{W}$ are trainable parameters, and $\hat{y}$ is the predict cruise control state.

2. Implement a neural network class `ACCNet` and associated training class. Since ACC state might by highly imbalanced, choose Dice Loss or Focal Tversky Loss as your training metrics. You are free to choose any kind of Neural Network architecture like. You can experiment with different style of architecture like ResNet, FourierNet, etc. or any custom layer for feature engineering, etc. However, you must explain your logic with necessary diagrams or mathematical equations. Custom layer must be implemented as its own class.

   There is **20 Points** bonus for achieve $> 90\%$ of validation accuracy.

3. Implement associated implementation script (python and bash scripts) in `script` folder of your package.

4. You are free to normalize dataset in any way you want use min-max scalar, standard normal scaler, etc.

5. Use most available GPU for training using GPU device selection method specified in the previous problem.

6. Save the trained model as ONNX along with normalizing coefficients to test with an unseen dataset. Commit ONNX file to your GitHub.